

Correction des exercices sur la logique de Hoare

Enseignement à distance de l'Université Marie et Louis Pasteur
Spécification et preuve de programmes

Version du 9 décembre 2025

1 Tableau de preuve en logique de Hoare

Comme toute preuve dans un système déductif, une preuve en logique de Hoare peut être présentée sous la forme d'un tableau à 3 colonnes. La première colonne sert à numéroter les formules, à partir de 1. La deuxième colonne est une formule obtenue selon l'explication donnée dans la troisième colonne. Cette explication est

- un axiome, ou
- un théorème considéré comme connu (autorisé dans l'énoncé de l'exercice), ou
- le nom d'une règle d'inférence du système déductif, suivi des numéros des formules précédentes auxquelles cette règle est appliquée.

Reproduire et compléter le tableau suivant pour qu'il soit une preuve complète sous forme de tableau du programme de calcul de la factorielle vu dans le cours sur la logique de Hoare.

No	Triplet	Justification
1	$\{i < n \wedge r = i!\} \ i := i + 1 \ \{i \leq n \wedge r = (i - 1)!\}$	(assignment)
2	$\{i \leq n \wedge r = (i - 1)!\} \ r := r * i \ \{i \leq n \wedge r = i!\}$	(assignment)
3	$\{i < n \wedge i \leq n \wedge r = i!\} \ i := i + 1; \ r := r * i \ \{i \leq n \wedge r = i!\}$	(sequence) 1 2
4	...	...
5	...	...
...	...	...

Avec la formule $i \leq n \wedge f = i!$ pour l'invariant I , le cours donne sous forme d'arbres une preuve complète du triplet de Hoare

$$\{n \geq 0\} \ i := 0; \ f := 1; \ \text{while } i < n \text{ do } i := i + 1; \ f := f * i \ \text{od } \{f = n!\}.$$

La solution suivante présente la même preuve dans un tableau.

No	Triplet	Justification
1	$\{i < n \wedge r = i!\} \ i := i + 1 \ \{i \leq n \wedge r = (i - 1)!\}$	(assignment)
2	$\{i \leq n \wedge r = (i - 1)!\} \ r := r * i \ \{i \leq n \wedge r = i!\}$	(assignment)
3	$\{i < n \wedge i \leq n \wedge r = i!\} \ i := i + 1; \ r := r * i \ \{i \leq n \wedge r = i!\}$	(sequence) 1 2
4	$n \geq 0 \wedge i = 0 \wedge r = 1 \Rightarrow I$	OK
5	$\{I\} \text{ while } i < n \text{ do } i := i + 1; \ r := r * i \text{ od } \{\neg(i < n) \wedge I\}$	(while) 3
6	$i \not\leq n \wedge I \Rightarrow r = n!$	OK
7	$\{n \geq 0 \wedge i = 0\} \ r := 1 \ \{n \geq 0 \wedge i = 0 \wedge r = 1\}$	(assignment)
8	$\{n \geq 0 \wedge i = 0 \wedge r = 1\} \text{ while } i < n \text{ do } i := i + 1; \ r := r * i \text{ od } \{r = n!\}$	(cons.) 4 5 6
9	$\{n \geq 0\} \ i := 0 \ \{n \geq 0 \wedge i = 0\}$	(assignment)
10	$\{n \geq 0 \wedge i = 0\} \ r := 1; \text{ while } i < n \text{ do } i := i + 1; \ r := r * i \text{ od } \{r = n!\}$	(sequence) 7 8
11	$\{n \geq 0\} \ i := 0; \ r := 1; \text{ while } i < n \text{ do } i := i + 1; \ r := r * i \text{ od } \{r = n!\}$	(sequence) 9 10

Remarque : La 3^e ligne du tableau comporte la précondition $\{i < n \wedge i \leq n \wedge r = i!\}$ tandis que la précondition de la première ligne est $\{i < n \wedge r = i!\}$. D'après la règle pour la composition séquentielle, en toute rigueur, ces deux formules devraient être syntactiquement identiques. On commet ici une petite approximation, qu'on peut éliminer en remplaçant la ligne 1 par les quatre lignes suivantes :

No	Triplet	Justification
-2	$(i < n \wedge i \leq n \wedge r = i!) \Rightarrow (i + 1 \leq n \wedge r = ((i + 1) - 1)!)$	OK
-1	$\{i + 1 \leq n \wedge r = ((i + 1) - 1)!\} \ i := i + 1 \ \{i \leq n \wedge r = (i - 1)!\}$	(assignment)
0	$(i \leq n \wedge r = (i - 1)!) \Rightarrow (i \leq n \wedge r = (i - 1)!)$	OK
1	$\{i < n \wedge i \leq n \wedge r = i!\} \ i := i + 1 \ \{i \leq n \wedge r = (i - 1)!\}$	(consequence) -2 -1 0

2 Deux règles supplémentaires

Dans son article fondateur, T. Hoare ne propose pas l'unique règle de conséquence mais les deux règles suivantes :

$$\frac{P \Rightarrow P' \quad \{P'\} c \{Q\}}{\{P\} c \{Q\}} \text{ (precondition)} \qquad \frac{\{P\} c \{Q'\} \quad Q' \Rightarrow Q}{\{P\} c \{Q\}} \text{ (postcondition)}$$

1. Expliquer clairement et simplement pourquoi ces deux règles peuvent être ajoutées à la logique de Hoare sans modifier ses propriétés de correction et de complétude.

Puisque $Q \Rightarrow Q$ est vraie pour toute formule Q (réflexivité de l'implication), ces deux règles sont des cas particuliers (= se démontrent à l'aide) de la règle de conséquence. Par conséquent, si la logique de Hoare est correcte (= ne démontre que des triplets valides) alors la logique de Hoare augmentée par ces deux règles est correcte : on peut remplacer toute déduction selon ces règles par une déduction selon la règle de conséquence. On devra seulement démontrer en plus une formule de la forme $Q \Rightarrow Q$.
Puisque la logique de Hoare est complète (= démontre tous les triplets valides) alors elle reste évidemment complète si on lui ajoute des règles, quelles qu'elles soient.

2. Même question si ces deux règles **remplacent** la règle de conséquence dans cette logique.

Puisque la logique de Hoare est correcte (= ne démontre que des triplets valides), elle reste évidemment correcte si on lui enlève des règles, quelles qu'elles soient. De plus, nous avons vu dans la réponse à la question 1 que les règles (*precondition*) et (*postcondition*) sont des cas particuliers de la règle (*consequence*), donc sont correctes car on a admis que la règle (*consequence*) est correcte. Par conséquent, l'ajout des règles (*precondition*) et (*postcondition*) à la logique de Hoare privée de la règle de conséquence constitue encore une logique correcte.

La règle de conséquence peut être démontrée comme suit à partir de ces deux règles :

$$\frac{\frac{P \Rightarrow P' \quad \{P'\} c \{Q\}}{\{P\} c \{Q'\}} \text{ (precondition)} \quad Q' \Rightarrow Q}{\{P\} c \{Q\}} \text{ (postcondition)}$$

Par conséquent, puisque la logique de Hoare est complète (= démontre tous les triplets valides), la logique où ces deux règles remplacent la règle de conséquence est complète : dans une preuve selon la logique de Hoare, remplacer toute déduction selon la règle de conséquence par la bonne instance de la preuve précédente. On obtient une preuve dans cette nouvelle logique.

Dans tous les exercices suivants, il est permis d'utiliser ces deux règles supplémentaires.

3 Racine carrée entière par incrémentation

Soit n un entier naturel. La *racine carrée entière par défaut* de n est le plus grand entier naturel r dont le carré est inférieur ou égal à n .

1. Spécifier par une précondition et une postcondition tout programme qui calcule dans une variable r la racine carrée entière par défaut d'un entier n .

Précondition : $n \geq 0$.
Postcondition : $r^2 \leq n < (r + 1)^2$.

2. Dans le langage de Hoare, donner un programme conforme à cette spécification qui procède (naïvement) par incrémentation de r .

$r := 0$; while $(r + 1)^2 \leq n$ do $r := r + 1$ od

3. En appliquant la logique de Hoare règle par règle, donner dans un tableau une démonstration formelle de la correction partielle de ce programme par rapport à cette spécification.

Dans la logique de Hoare, une démonstration de validité du triplet de Hoare

$$\{n \geq 0\} \ r := 0; \text{while } (r+1)^2 \leq n \text{ do } r := r+1 \text{ od } \{r^2 \leq n < (r+1)^2\}$$

est :

N°	Triplet ou formule	Justification
1	$(r+1)^2 \leq n \wedge r^2 \leq n \Rightarrow (r+1)^2 \leq n$	OK
2	$\{(r+1)^2 \leq n\} \ r := r+1 \ \{r^2 \leq n\}$	(assignment)
3	$\{(r+1)^2 \leq n \wedge r^2 \leq n\} \ r := r+1 \ \{r^2 \leq n\}$	(precond. 1 2)
4	$n \geq 0 \wedge r = 0 \Rightarrow r^2 \leq n$	OK
5	$\{r^2 \leq n\} \text{ while } (r+1)^2 \leq n \text{ do } r := r+1 \text{ od } \{(r+1)^2 \not\leq n \wedge r^2 \leq n\}$	(while) 3
6	$\neg((r+1)^2 \leq n) \wedge r^2 \leq n \Rightarrow r^2 \leq n < (r+1)^2$	OK
7	$\{n \geq 0\} \ r := 0 \ \{n \geq 0 \wedge r = 0\}$	(assignment)
8	$\{n \geq 0 \wedge r = 0\} \text{ while } (r+1)^2 \leq n \text{ do } r := r+1 \text{ od } \{r^2 \leq n < (r+1)^2\}$	(cons.) 4 5 6
9	$\{n \geq 0\} \ r := 0; \text{while } (r+1)^2 \leq n \text{ do } r := r+1 \text{ od } \{r^2 \leq n < (r+1)^2\}$	(sequence) 7 8

Les lignes marquées OK méritent aussi une justification (en dehors du tableau), à rédiger.

4 Instruction conditionnelle

1. Utiliser la commande `if ... then ... fi` pour définir une commande qui calcule dans une variable x la valeur absolue d'un nombre stocké dans la même variable x .

La commande

`if  $x < 0$  then  $x := -x$  fi`

calcule la valeur absolue de x dans la variable x .

2. Définir la correction de ce programme à l'aide d'un triplet de Hoare.

Ce programme est correct si et seulement si le triplet de Hoare

$$\{x = y\} \text{ if } x < 0 \text{ then } x := -x \text{ fi } \{x = |y|\}$$

est valide. Ce triplet utilise une variable supplémentaire y , appelée "variable de spécification", ou "variable fantôme" car elle n'est pas utilisée dans le programme.

La postcondition $x = |y|$ peut être considérée comme une abréviation de la formule

$$(y \geq 0 \Rightarrow x = y) \wedge (y < 0 \Rightarrow x = -y).$$

3. Donner sous forme de tableau une preuve formelle de la validité de ce triplet de Hoare.

N°	Triplet ou formule	Justification
1	$x < 0 \wedge x = y \Rightarrow -x = y $	OK
2	$\{ -x = y \} x := -x \{ x = y \}$	(assignment)
3	$\{ x < 0 \wedge x = y \} x := -x \{ x = y \}$	(precondition)
4	$(\neg(x < 0) \wedge x = y) \Rightarrow x = y $	OK
5	$\{ x = y \} \text{ if } x < 0 \text{ then } x := -x \text{ fi } \{ x = y \}$	(if-then) 3 4

Après élimination de l'abréviation de la postcondition, la formule 1 est :

$$x < 0 \wedge x = y \Rightarrow (y \geq 0 \Rightarrow -x = y) \wedge (y < 0 \Rightarrow -x = -y)$$

L'hypothèse $x = y$ permet de remplacer y par x dans sa conclusion. Il reste donc à démontrer

$$x < 0 \Rightarrow (x \geq 0 \Rightarrow -x = x) \wedge (x < 0 \Rightarrow -x = -x)$$

qui est évident.

Après élimination de l'abréviation de la valeur absolue, la formule 4 est :

$$x \geq 0 \wedge x = y \Rightarrow (y \geq 0 \Rightarrow x = y) \wedge (y < 0 \Rightarrow x = -y)$$

De même, l'hypothèse $x = y$ permet de remplacer y par x dans sa conclusion. Il reste donc à démontrer

$$x \geq 0 \Rightarrow (x \geq 0 \Rightarrow x = x) \wedge (x < 0 \Rightarrow x = -x)$$

qui est évident.